

BasicAttackMontage 如何變成一次出拳？

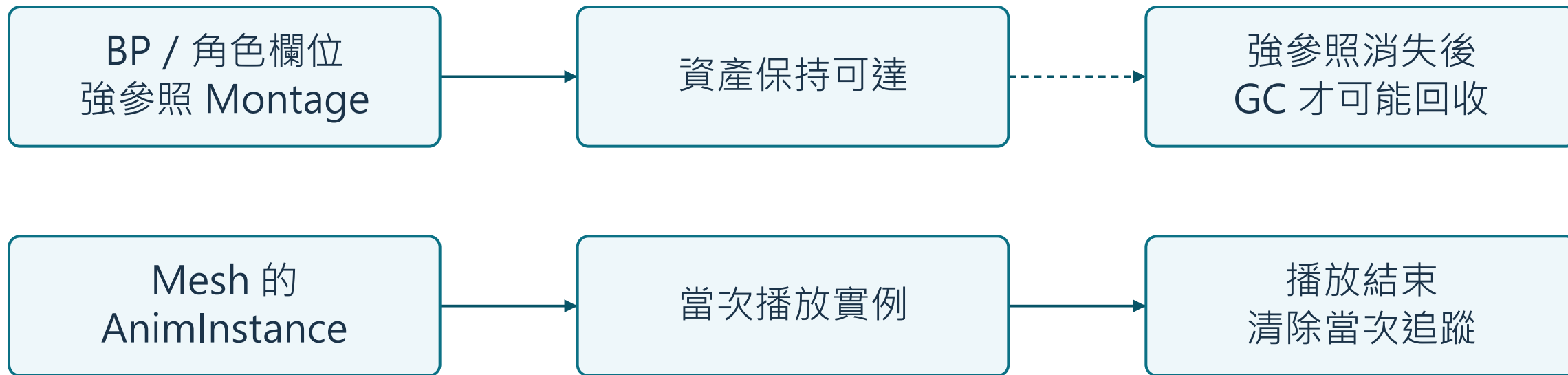


BasicAttackMontage 是資產參照；FAnimMontageInstance 才是這次播放。

Montage 資產不會自行播放；BP 將它指定給角色欄位，C++ 通過檢查後交給 AnimInstance 播放，Slot 才把姿勢接入畫面。

程式定位 | FightingCharacter.h:118–120；FightingCharacter.cpp:397–442 TryExecuteCommand()

UObject 何時載入、由誰持有、何時釋放？

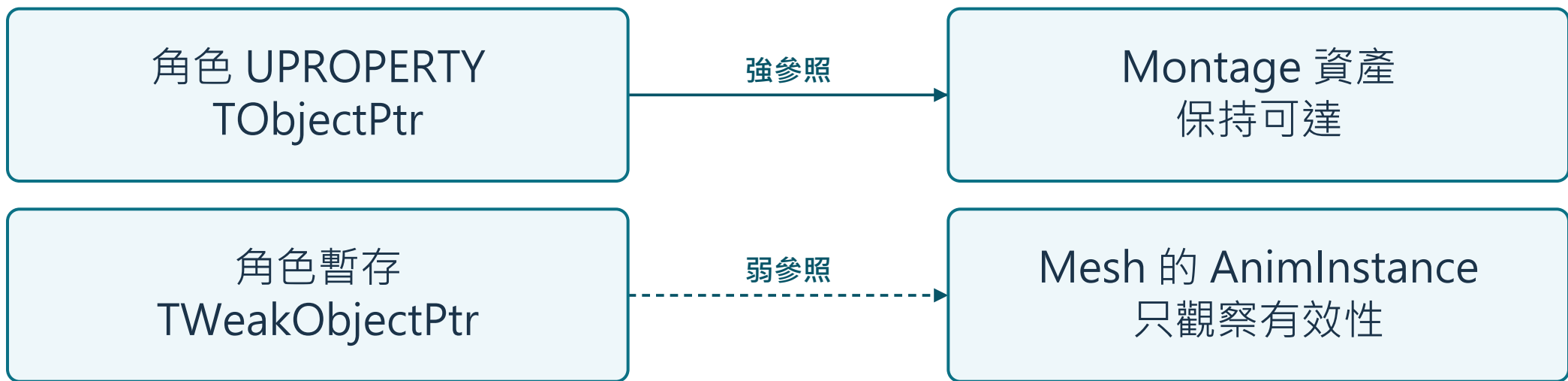


載入、播放結束、UObject 回收是不同時點；播放實例是 struct。

資產載入、播放實例結束與 UObject 回收是三個不同時點；播放收尾不等於立刻卸載 Montage 資產。

程式定位 | FightingCharacter.h:118, 139–144 ; FightingCharacter.cpp:397–442, 471–484

TObjectPtr 與 TWeakObjectPtr：誰讓目標保持可達？



UPROPERTY 讓 GC 看見強參照；弱參照也可以標 UPROPERTY。

‘UPROPERTY’ 搭配 ‘TObjectPtr’ 讓 GC 看見角色對資產的強參照，也把欄位公開給 Class Defaults；弱參照只記錄當次播放者，不延長其生命週期。

程式定位 | FightingCharacter.h:118–120 BasicAttackMontage；:139–144 AttackAnimInstance

結束回呼要認出是哪一次播放

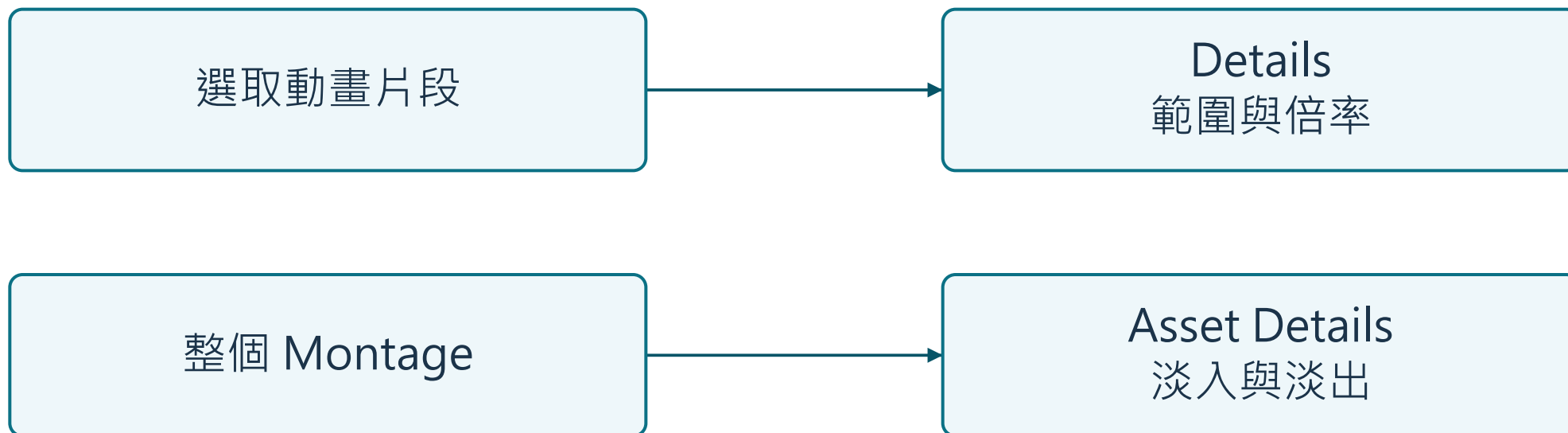


AnimInstance 與 instance ID 是本專案附帶的身分，不是 BindUObject 的必填參數。

‘BindUObject’ 綁定的是角色接收函式；‘AttackAnimInstance’ 和 ‘AttackInstanceId’ 是額外附帶的當次播放身分，不是 ‘BindUObject’ API 一律要求的參數。

程式定位 | FightingCharacter.cpp:430–434 BindUObject() ; :460–484 HandleAttackEnded() / FinishAttack()

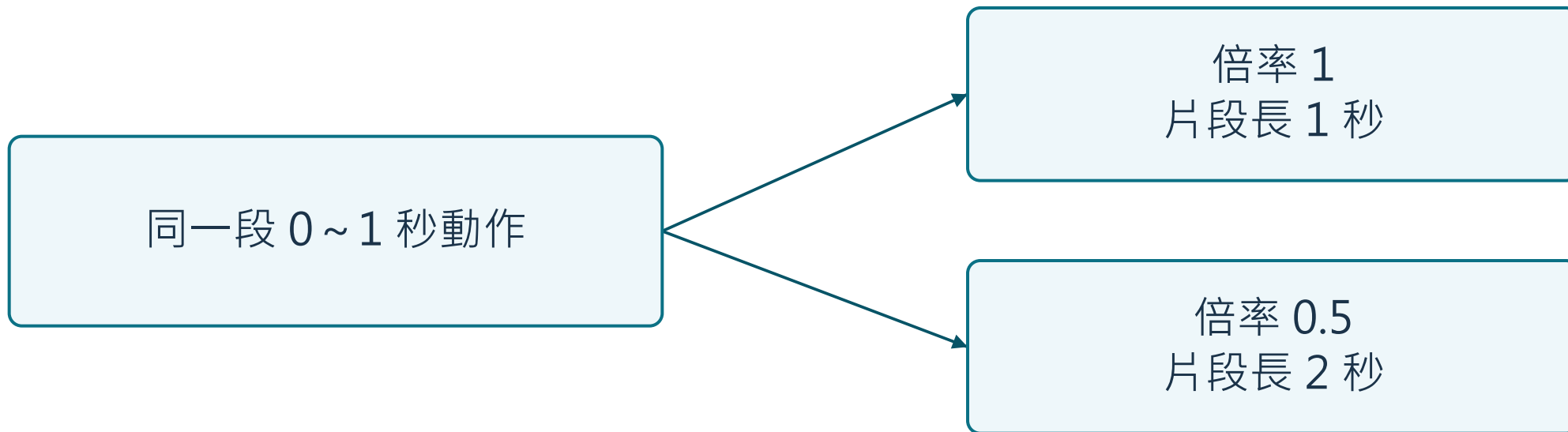
在時間軸選對編輯對象



片段 Details 與 Montage 的 Asset Details，控制的是不同層級。

程式定位 | AM_Attack_01 片段 Details / Asset Details ; FightingCharacter.cpp TryExecuteCommand()

Play Rate 改的是動作播放速度



片段時長 = 使用範圍 ÷ 片段倍率；解鎖依播放結束通知。

半速播放會拉長片段時間，角色解鎖仍要等實際播放完整結束。

程式定位 | AM_Attack_01 片段 Play Rate ; FightingCharacter.cpp TryExecuteCommand() / Montage_Play()

Blend In / Out 改的是姿勢過渡

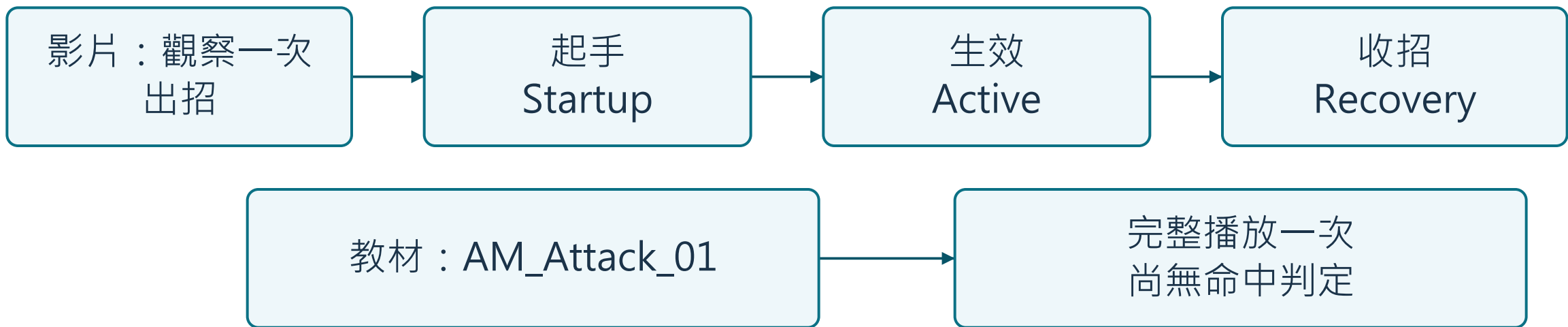


正常淡入：Montage_Play + Blend In。自然淡出：Auto Blend Out。提前取消：Stop(BlendOut)。

角色程式要求開始播放，Montage 的混合設定交由引擎套用；`DefaultGroup.DefaultSlot` 在攻擊姿勢與 `Main States` 傳入的原本姿勢之間切換權重。

程式定位 | TryExecuteCommand(): Montage_Play ; AM_Attack_01: Auto Blend Out ; CancelAttack(): Stop(BlendOut)

遊戲畫面對照：起手、生效與收招

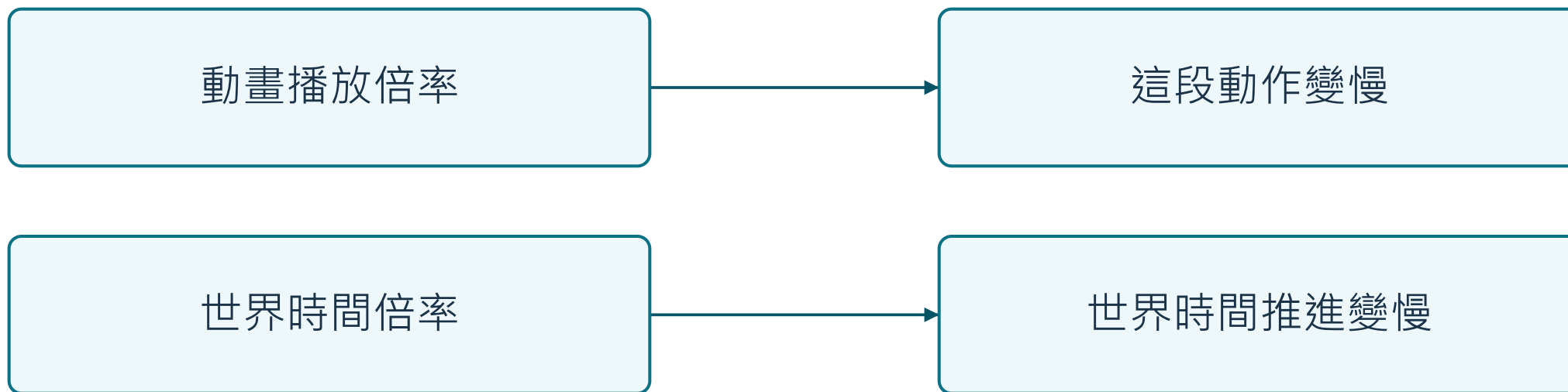


Startup / Active / Recovery 是觀察用詞，本週尚未建立對應命中規則。

用起手、生效、收招觀察外部遊戲的出招節奏；教材的 `AM\_Attack\_01` 目前只負責完整播放，沒有設定命中生效窗口。

程式定位 | FightingCharacter.cpp:417 Montage_Play(); 尚無命中窗口或 Hitbox 程式

遊戲畫面對照：慢動作改在哪一層？

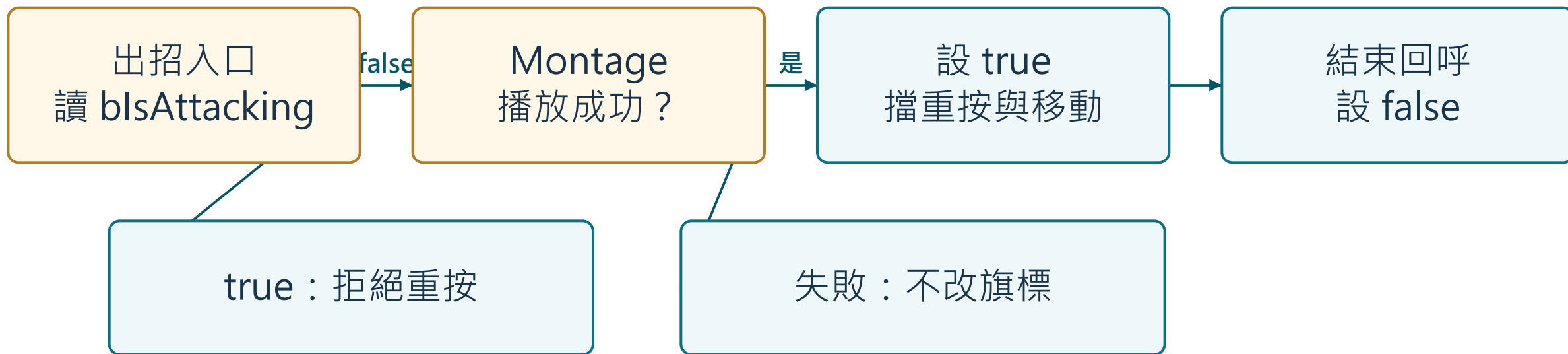


影片只供觀察；無法由畫面反推《TEKKEN 8》的慢動作實作。

教材修改的是 Montage 片段的 Play Rate；影片只能用來觀察慢動作效果，不能據此判定《TEKKEN 8》採用哪種實作。

程式定位 | AM_Attack_01 片段 Play Rate；FightingCharacter.cpp:417 Montage_Play(..., 1.0f)

沿流程讀懂 blsAttacking 如何鎖住與放行

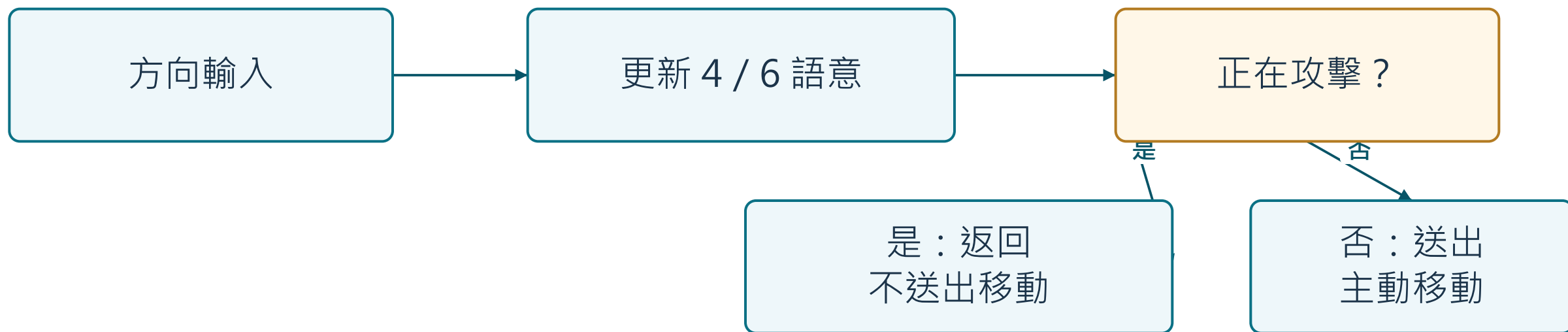


下一次按 X 再回到入口評估；這只是布林旗標，沒有獨立攻擊狀態機。

同一個 ``blsAttacking`` 由出招入口檢查、播放成功後設置、移動入口讀取，並在當次播放結束後清除；下一次出招再回到入口檢查。

程式定位 | `FightingCharacter.cpp:397–434, 460–479` ; `FightingPlayerCharacter.cpp:134–175 DoMove()`

鎖住動作，仍然讀取方向

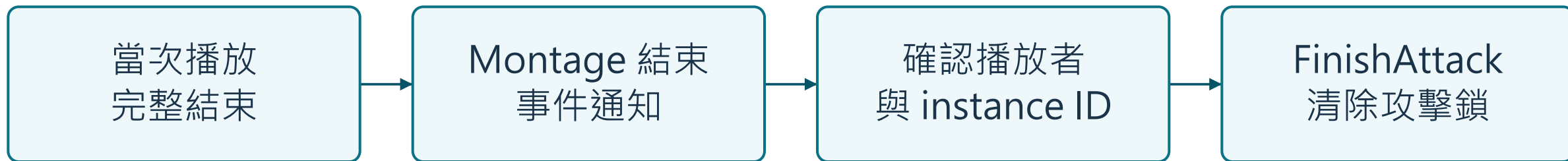


4 / 6 是輸入意圖；外部碰撞、重力與距離修正沒有全部停用。

攻擊中保留輸入語意，但擋住主動移動與自動面向。

程式定位 | FightingPlayerCharacter.cpp DoMove() ; FightingCharacter.cpp UpdateFacing()

把結束通知接回角色

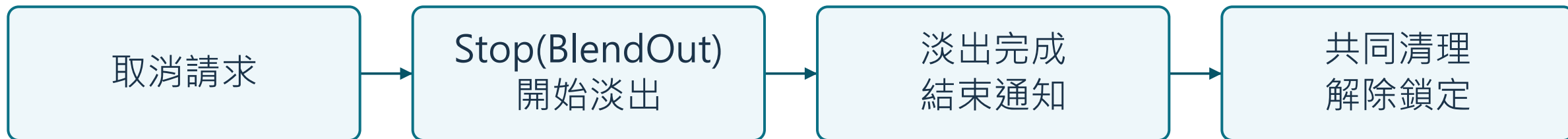


收到完整結束通知後才清旗標；沒有固定秒數解鎖 Timer。

回呼屬於這次播放，清理也只處理這次攻擊。

程式定位 | FightingCharacter.cpp BindUObject() → HandleAttackEnded() → FinishAttack(false)

取消只開始淡出，不立即解鎖

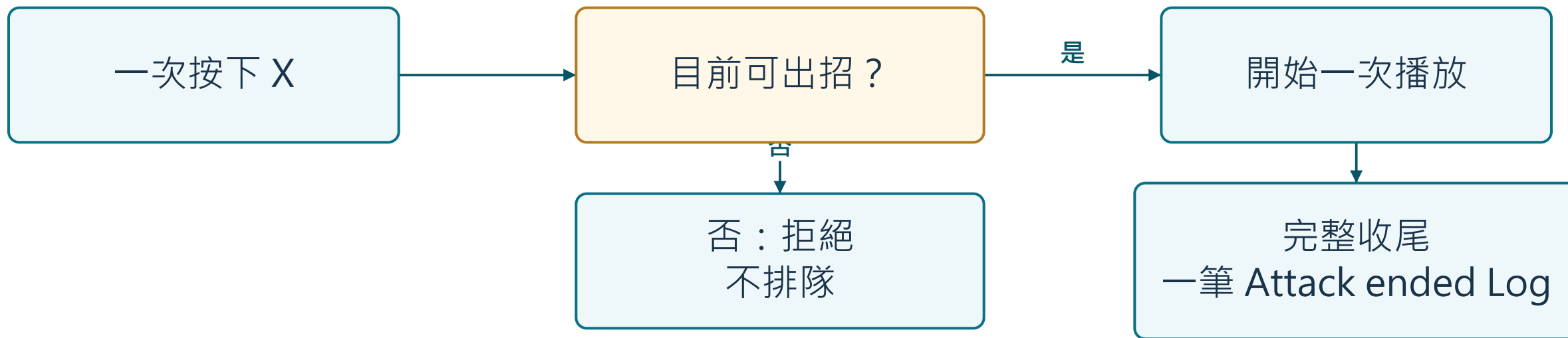


CancelAttack() 不直接把 blsAttacking 設回 false；淡出期間仍保持鎖定。

取消與正常播放都走到完整收尾，不能在開始淡出時先放行。

程式定位 | FightingCharacter.cpp CancelAttack() → HandleAttackEnded() → FinishAttack()

重按不排隊，Log 記的是收尾

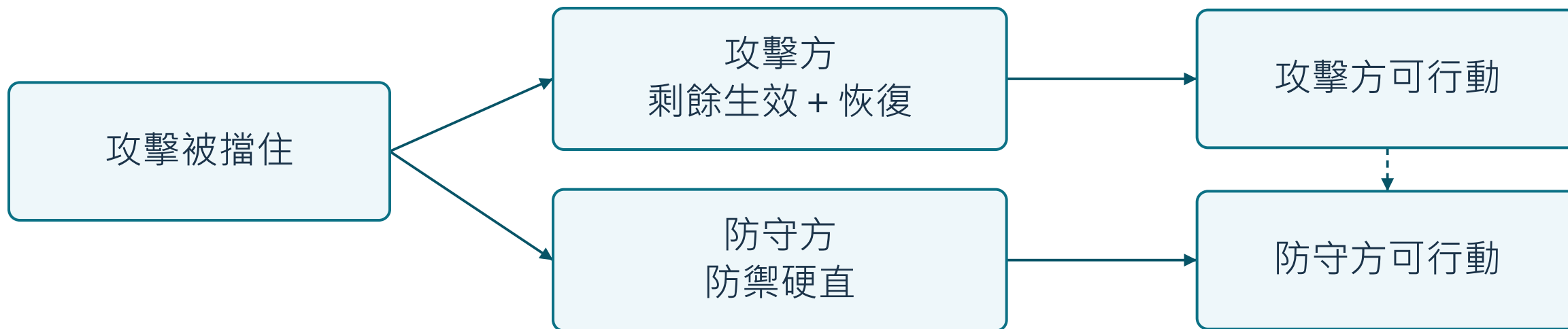


只把既有 Attack ended 的 UE_LOG 等級由 Verbose 改成 Log。

只有被接受的播放才有結束事件；按鍵次數不等於收尾次數。

程式定位 | FightingPlayerCharacter.cpp Attack() ; FightingCharacter.cpp HandleAttackEnded()

遊戲畫面對照：被擋後 +5F



Frame Advantage = 防守方可行動幀 - 攻擊方可行動幀

+5F：攻擊方早 5 幀恢復 -5F：防守方早 5 幀恢復

比較同一次接觸後的恢復時點；圖線未依幀數比例繪製。

以 +5F 教學假設比較雙方恢復行動的時間；這不是教材專案目前會計算的數值。

程式定位 | 本週無防禦或幀優勢計算；僅對照 FightingCharacter.cpp:460–479 的單方收尾